

8. Flash Sale — Fixed Inventory, Extreme Demand

Interviewer: Amazon is dropping a new iPhone at noon — **5,000 units, but 2 million people** want it and they're all refreshing at 11:59. Design the system that runs this flash sale: it must never oversell, must feel fair, and must not take down the rest of Amazon. How would you build it?

Candidate: Great problem — this is the "hot product" pattern. The interesting tension is **tiny fixed inventory + enormous concurrent demand + a real payment step**, versus the burger giveaway which was free. Let me clarify first.

1. Clarifying requirements

Candidate: 1. Fixed inventory — **5,000 units, hard cap, never oversell** even by one? 2. There's a **checkout + payment** step, unlike a free giveaway — so "reserving" a unit and "paying for it" are two phases. Do we hold inventory during checkout? 3. What's "fair" — first-come-first-served, or a lottery? FCFS is more common for Amazon. 4. Does the *rest of Amazon* (normal catalog, other products) have to stay healthy while this sale melts one product page? (I assume yes — blast-radius isolation.)

Interviewer: Hard cap 5,000, never oversell. Yes — reserve on "Add to cart / Buy now", then a short window to pay or the unit is released. FCFS. And absolutely — this one product must not degrade the rest of the site.

Candidate: Perfect. Requirements:

Functional - Show product; accept "Buy now" → **reserve** a unit (a temporary hold). - Reserved user gets a **checkout window** (e.g. 2–5 min) to pay. - On payment success → unit is **sold**. On timeout/abandon → unit **returns to pool**. - Never sell more than 5,000. Never double-charge.

Non-functional - **Strong consistency on inventory** — 5,000 is a hard ceiling. - **Massive read spike** on the product page (2M refreshers) + **write spike** on "Buy now". - **Isolation:** the flash sale must be **firewalled** from the main commerce platform. - **Fairness + graceful rejection:** the 1.995M who lose should get a fast, clean "sold out", not a spinner or a 500.

2. Estimation

2M interested users, most active in the first ~2 minutes.

Reads (product page, "is it live?", "any left?"):

assume each user refreshes ~10x → 20M reads in ~120s → ~170k reads/sec (peak higher). This is a READ-heavy storm on ONE product.

Writes ("Buy now" taps): maybe 2M taps in the first 30s → ~65k write-attempts/sec

...all contending for 5,000 units. 99.75% of them must be rejected fast.

So: 170k+/s reads on one item AND 65k/s writes on a 5,000-deep pool.

The reads we solve with caching + a waiting room; the writes we solve with an atomic, sharded reservation counter – same family as the burger problem, but now with a two-phase reserve→pay and a real oversell-is-catastrophic requirement.

3. The reads: cache + waiting room (protect the platform)

Candidate: First, the 170k/s of reads must never hit the primary inventory system. The product page is essentially static during the sale.

```
[ 2M clients refreshing ]
|
[ CDN ] — product page, images, "sale starts at noon" = fully cached, ~0 origin load
|
[ Waiting Room / Virtual Queue ] — issues a queue token; admits users to the
|                                "Buy" backend at a controlled rate (e.g. 5–10k/s)
[ API Gateway (L7) + rate limiting ]
|
[ Flash-sale service (ISOLATED cluster) ]
```

- **CDN** absorbs the read storm — the page, the countdown, the "in stock / sold out" badge are cached with a short TTL (1–2s). 170k/s of reads become ~a handful of origin fetches.
- **Waiting room** is the crucial fairness + protection layer. Users get a queue position; the backend only admits people at a rate it can actually process. This converts a 65k/s write spike into a smooth, survivable trickle and gives users an honest "you're #48,213 in line" instead of a frozen app. (This is exactly how real ticket/drop systems survive.)
- **Isolation:** the flash-sale service runs on its **own cluster with its own datastore**, separate from the main Amazon commerce stack. If it melts, the rest of the catalog is untouched. This is the blast-radius requirement made concrete.

4. The writes: atomic reservation, never oversell

Interviewer: Someone gets admitted and taps "Buy now." Walk me through reserving a unit without overselling.

Candidate: Same core trick as any fixed-inventory problem: **an atomic decrement on a fast in-memory counter, pre-loaded with the 5,000 units — not a database row lock.**

```
Buy now → reserved = DECR stock:{shard}          (atomic in Redis)
          |
          |— reserved >= 0 → create a RESERVATION (hold) with TTL, return "checkout"
          |— reserved < 0 → (probe siblings) → else SOLD_OUT (fast, clean)
```

- Inventory of 5,000 is small, so I might shard into, say, **10 counters of 500** to spread the 65k/s across keys, with sibling-probing near the end so we truly sell all 5,000.
- **Reservation = a temporary hold**, not a sale. We write a `reservation:{userId}` key in Redis with a **TTL equal to the checkout window** (say 3 min). This is the key difference from the burger problem — there's a *second phase*.
- **Per-user guard** (`SET NX reserved:{userId}`) so one user can't grab multiple units by double-tapping.

The reserve → pay → confirm state machine

```
[AVAILABLE] —DECR ok→ [RESERVED (TTL 3m)] —pay success→ [SOLD]
                        |
                        |— TTL expires / user abandons → INCR stock (release)
                                                           back to [AVAILABLE]
```

Why the TTL hold matters: without it, users who reserve but never pay would permanently lock units, and we'd "sell out" with unpaid holds — the classic ticketing bug. The TTL auto-returns abandoned units to the pool. A sweeper (or Redis key-expiry callback) does the `INCR` on expiry.

5. Payment — off the critical path, idempotent

Interviewer: They reserved a unit. Now the payment. This is money — what changes?

Candidate: Payment is where correctness gets strict, so:

```
[Reserved user] → POST /checkout (Idempotency-Key)
                |
                |— validate the reservation still holds (Redis key present, owned by this user)
                |— call Payment Service (idempotent — retries never double-charge)
                   success → emit "order-confirmed" to Kafka
                   fail    → release the hold (INCR stock), tell user
                ↓
[ Kafka: order-events ] → [ workers ] → [ Orders DB (source of truth) ]
                                ↳ [ email/SMS confirmation ]
```

- **Idempotency key on /checkout** — a flaky-network retry of "Pay" must not charge twice or create two orders. Server stores `(idempotency_key → result)` and returns the original result on a duplicate.
- **The durable order write is async** behind Kafka, exactly like the burger audit log — the user gets "order confirmed" as soon as payment + reservation are validated; the Orders DB row and the

shipping/notification pipeline happen behind the queue.

- **Payment is the one strongly-consistent external call** — we treat its success as the commit point that flips `RESERVED` → `SOLD`.

6. Technology choices — what and why (the detailed part)

Concern	Choice	Why this, and why not the alternative
Read storm on the product page	CDN + short-TTL cache	170k/s of identical reads collapse to a few origin hits; the page is effectively static during the sale. <i>Not hitting the DB per read</i> — that would take down the item.
Fairness + spike flattening	Virtual waiting room (token queue)	Turns 65k/s of uncontrolled taps into an admit rate the backend can serve; gives honest queue feedback; prevents thundering herd.
Reservation counter	Redis (sharded atomic counters)	Atomic <code>DECR</code> , memory-speed, no oversell within a key, shardable to spread contention. <i>Not <code>SELECT FOR UPDATE</code> on a PostgreSQL stock row</i> — one hot row serializes 65k/s of writers and collapses throughput.
The hold with a timeout	Redis key with TTL	Auto-releases abandoned reservations back to the pool with zero extra machinery; TTL == checkout window.
Per-user limit	Redis <code>SET NX</code>	O(1), shards by user, no hot key.
Orders (source of truth)	Relational DB (PostgreSQL / Amazon Aurora)	Orders are money + relationships (user, payment, shipment) needing ACID transactions and strong consistency; volume (5,000 orders) is trivial. Written async behind Kafka so DB latency never gates checkout.
Event backbone	Kafka	Durable, replayable, partitioned stream for order events → orders DB, fulfillment, notifications, analytics; decouples the sale from downstream. <i>Not synchronous calls</i> — a slow warehouse service must not slow checkout.
Payment	Dedicated Payment Service, idempotent	Isolates money handling; idempotency keys prevent double charges on retry.
Isolation	Separate flash-sale cluster + datastore	Blast-radius containment — a meltdown on the iPhone drop cannot degrade the rest of Amazon.

Say-it-out-loud justification: "Redis is the atomic, shardable reservation engine that guarantees we never oversell 5,000 units at 65k writes/sec; a CDN plus a virtual waiting room absorb the 170k/s read storm and flatten the write spike into a rate we can serve; the relational Orders DB is the ACID source of truth for money, written asynchronously behind Kafka so its latency never touches the buyer; and the whole flash-sale service is isolated on its own cluster so it can never take down the rest of the platform."

7. Consistency model — what's strong, what's eventual

- **Strong:** the reservation counter (never oversell) and the payment→order commit. Redis atomic ops guarantee the count; payment success is the linearization point for a sale.
- **Eventual:** the product-page "stock remaining" badge (cached 1–2s — approximate is fine), the Orders-DB row, and the confirmation email (all async behind Kafka).
- **The reserve-vs-sold gap:** units in `RESERVED` are neither available nor sold. We deliberately treat a reservation as "spent" for availability (so we don't oversell) but not yet as revenue. TTL expiry reconciles the gap by returning abandoned holds. This is read-your-writes for the buyer (they see their own reservation) and eventual for everyone else's view of remaining stock.

8. Failure modes & graceful degradation

Failure	What happens	Mitigation
Redis shard down	Its units stranded; some reservations lost	Replicas + failover; sibling-probe routes around it; reconciliation returns stranded stock
Payment service slow/down	Checkouts stall	Reservation TTL protects inventory (auto-release); user told to retry; Kafka buffers confirmed orders
Orders DB down	Order rows delayed	Kafka retains order events; workers drain on recovery — no lost orders, buyer already confirmed
Waiting room bypass attempt	Bots skip the queue	Signed queue tokens required at the gateway; per-device/IP limits; CAPTCHA
Sale overwhelms cluster	—	Isolated cluster + admission control; worst case the <i>sale</i> degrades, not the rest of Amazon

9. Follow-up curveballs

Interviewer: Two users reserve the *last* unit at the same microsecond. What happens?

Candidate: They don't — that's the whole point of the atomic `DECR`. Redis is single-threaded per key, so the two `DECR`s are strictly ordered: one gets `0` (granted), the other gets `-1` (sold out). There is no window where both succeed. That's why we push the scarcity decision into one atomic op instead of a read-modify-write in application code.

Interviewer: A reservation TTL expires *while* the user is on the payment page. They pay a second later. Now what?

Candidate: The `/checkout` handler **re-validates the reservation before charging**. If the hold expired and the unit was already re-sold to someone else, we **decline before taking money** and show "sorry, your hold expired." If the unit is still free (nobody grabbed the released unit yet), we can re-`DECR` and

honor it. The invariant is: **never charge without a valid held unit at charge time**. This is why payment validates the hold as its first step.

Interviewer: Scalpers using bots buy everything. How do you make it fairer?

Candidate: Layer defenses: per-account limit via `SET NX`, per-device + per-IP rate limits at the gateway, signed waiting-room tokens (can't skip the queue), CAPTCHA/bot-detection in the waiting room, and optionally account-age/history gating. Perfect fairness is impossible, but these raise the cost of abuse a lot.

Interviewer: Marketing wants a live "3,201 sold" counter.

Candidate: Same as the burger case — never read the hot inventory key at 170k/s. Compute `sold = 5000 - sum(stock shards)` on a 1–2s timer in the background, cache it, serve via CDN. Cosmetic, approximate, cheap — kept far away from the real inventory keys.

10. Deep-dive: "Why not just decrement real inventory in the primary DB at reserve time?"

Interviewer: All this Redis machinery — sharded counters, `SET NX`, TTL holds — feels heavy. Why not just do the obvious thing: `BEGIN; SELECT stock FOR UPDATE; if stock > 0, UPDATE stock = stock - 1 and INSERT a reservation row; COMMIT`? It's one transaction, it's ACID, and now the DB is the single source of truth for both inventory *and* the reservation. Why add Redis at all?

Candidate: It's the same lock-contention story as the burger's naive V1, plus a second, sharper problem that's specific to flash sales: **the reservation has a lifecycle, and a database transaction has no native concept of "undo this in 3 minutes if nothing else happens."**

Problem 1 — the hot row can't survive 65k writes/sec

`SELECT ... FOR UPDATE` on the one `iphone` stock row means every one of the 65k/s "Buy now" taps serializes on that row's lock, even though each transaction only holds it for a millisecond or two:

```
65,000 requests/sec -> all want the SAME row lock
lock hold time ~1-2ms -> theoretical ceiling ~500-1,000 grants/sec
=> 64,000+ requests/sec queue up behind the lock, timing out long before
    they get a turn. Exactly the burger $3 failure, just with a $999 price tag
    instead of a free burger.
```

Sharding the row (10 counters of 500, like \$4) helps throughput, but you're now rebuilding Redis's atomic-counter behavior *inside Postgres*, with worse per-op latency (disk-backed WAL + lock manager vs. a single-threaded in-memory `DECR`).

Problem 2 — the reserved-but-unpaid inventory leak

Even if throughput were solved, a **DB row decrement has no expiry primitive**. Once you `UPDATE stock = stock - 1` and write a reservation row, that unit is gone from the pool *forever* unless something else notices the user never paid and reverses it:

User taps "Buy now" -> stock decremented, reservation row inserted, status = RESERVED
User closes the tab, never pays.
... nothing happens automatically. The unit is stuck in RESERVED.
A cron job must scan reservations WHERE status='RESERVED' AND created_at < now()-3min,
and issue a compensating UPDATE stock = stock + 1 for each one it finds.

That sweep runs on a schedule — every 30s, every minute — so an abandoned unit sits **idle and unsellable** for however long it takes the sweeper to notice, on top of the checkout window itself. At 65k/s of contention, a sweeper doing row-by-row compensating updates is *itself* new load on the same hot table. Redis's key `TTL` gives you the same "expire and reclaim" behavior as a **first-class, lock-free primitive** — no polling job, no extra writes to the hot row, reclaimed within milliseconds of expiry.

Concern	DB-transaction decrement (<code>FOR UPDATE</code>)	Redis reservation counter + TTL
Throughput at 65k/s	Collapses to ~500-1k/s; every writer serializes on one row lock	Atomic <code>DECR</code> , memory-speed, shardable to spread the 65k/s across keys
Hold expiry	No native concept — requires a polling sweeper job scanning <code>created_at</code>	Native key <code>TTL</code> ; expires and is reclaimed automatically, no polling
Reserved-but-unpaid leak	Unit is unsellable until the sweeper's next pass (extra minutes of idle inventory)	Unit returns to pool within ~ms of TTL expiry
Extra load from cleanup	Sweeper issues its own writes against the same hot table it's trying to protect	Expiry is handled by Redis itself; zero extra writes from the app
Where correctness lives	ACID transaction — strong, but paid for in serialized latency	Single-threaded atomic op per key — strong <i>and</i> fast
Operational cost	"Just SQL," but you end up hand-building a lock-free counter and a TTL system on top of a row store	One more stateful system to run (Redis), but the primitives you need come for free

Rule of thumb: *If a piece of state has to expire on its own, don't bolt an expiry system onto a row store — put it somewhere expiry is a first-class primitive.* The DB is still right for the *durable, ACID order record* once payment succeeds; it's the wrong place to park a temporary, self-expiring hold under extreme write contention.

11. Deep-dive: the reservation-expires-mid-payment race

Interviewer: Let's go deeper on something you mentioned earlier. A user's reservation TTL expires *while they're on the payment page*. The freed unit gets grabbed by someone else. Then — a second

later — the **first user's payment actually succeeds**. Walk me through exactly what goes wrong, and how you'd prevent it.

Candidate: This is the sharpest correctness edge in the whole design, because it isn't a throughput problem — it's a **race between three independent clocks**: the reservation TTL, the user's typing/network speed, and the payment gateway's response time. None of them know about each other unless we make them.

The timeline

```
t=0:00:00  User A reserves the LAST unit.
           DECR stock -> 0 (granted). reservation:A written, TTL = 180s.
           stock is now fully committed on paper: 0 available, 1 held by A.

t=0:02:55  A is still on the payment page (slow typing, distracted, spotty
           network). No request has left their browser yet.

t=0:03:00  TTL EXPIRES. reservation:A key vanishes from Redis.
           On-expiry hook: INCR stock -> 1 available again.
           From the system's point of view, the unit is back in the pool —
           A's hold is gone, but A's browser doesn't know that.

t=0:03:01  User B, who's been waiting/refreshing, taps "Buy now."
           DECR stock -> 0 (granted). reservation:B written, TTL = 180s.
           The unit is now legitimately B's.

t=0:03:05  User A finally submits payment. Their browser had the card form
           filled out BEFORE the TTL expired, so from A's perspective nothing
           looks wrong. POST /checkout for user A arrives at the server.

           — If /checkout blindly trusts "A had a reservation when they
             loaded the page" and charges A's card right now: —

           Payment succeeds. But the unit belongs to B.
           => EITHER we ship two orders for one unit (oversold),
           => OR we've charged A with nothing to give them
              (charge-without-item).
```

The bug isn't in the reservation logic — DECR / INCR /TTL all behaved exactly as designed. The bug is in **trusting a reservation that was true in the past without re-checking it at the moment we're about to take the user's money**.

Ranked fixes

1. **Re-validate the reservation at payment-capture time, before charging (mandatory, not optional).** /checkout must atomically check "does reservation:A still exist and is it still owned by A?" as the *very first step*, and only call the payment gateway if that check passes. If A's hold is gone, decline immediately with "sorry, your hold expired" — **zero dollars ever move**. This closes the timeline above at `t=0:03:05`: the check fails, we never call Payment Service, done.
2. **Compensating auto-refund if the unit is gone by the time the charge lands (safety net for the residual race).** Even with fix #1, there's a razor-thin window between "we checked the reservation" and "the payment gateway confirms the charge" — in theory the reservation could still flip in that gap under extreme concurrency. So after a successful charge, do one more

authoritative check that the reservation is still `A`'s and the order can be created; if not, **immediately issue an automatic refund** and notify the user, rather than silently keeping their money. This is the belt-and-suspenders move, not the primary defense.

3. **Make the TTL comfortably longer than the expected payment window.** If checkout normally takes 60-90 seconds, a 3-5 minute TTL shrinks how often this race even happens. Cheap and worth doing, but it's a *probability reducer*, not a fix — a slow enough user can always outrun any fixed TTL, so this can't be the only defense.
4. **Extend (renew) the hold the moment the user starts paying.** When the user lands on the payment page / submits a payment intent, atomically bump the reservation into a `PAYMENT_IN_PROGRESS` state with its own fresh timeout (e.g. re-`SET` the key with a new TTL, or `PEXPIRE` it). This means the clock only runs down on *idle browsing time before checkout starts*, not on time spent actively paying — which is exactly when we most want the hold to survive.

Recommended approach

Lead with **#1 as the non-negotiable invariant** — never call the payment gateway without a freshly-validated reservation, full stop. Pair it with **#4** so that in the common case a genuinely-paying user's hold simply doesn't expire out from under them. Keep **#2** as a safety net for the vanishingly rare gap between validation and gateway confirmation — you never want a world where "the payment succeeded" and "the customer got nothing and no refund" can both be true. Treat **#3** (a generous TTL) as a supporting tweak, not a strategy on its own.

What to say out loud: *"The invariant is: we never take money without a freshly validated hold. Re-checking the reservation at the top of the charge path — atomically, right before calling the payment gateway — is what actually prevents oversell and charge-without-item; a longer TTL and a payment-start renewal just make the race rarer. And if the charge and the loss of the hold ever land in the same narrow window despite all of that, an automatic refund is the last line of defense — a customer should never be left paying for a unit they don't have with no way to know it."*

Summary — the mental model

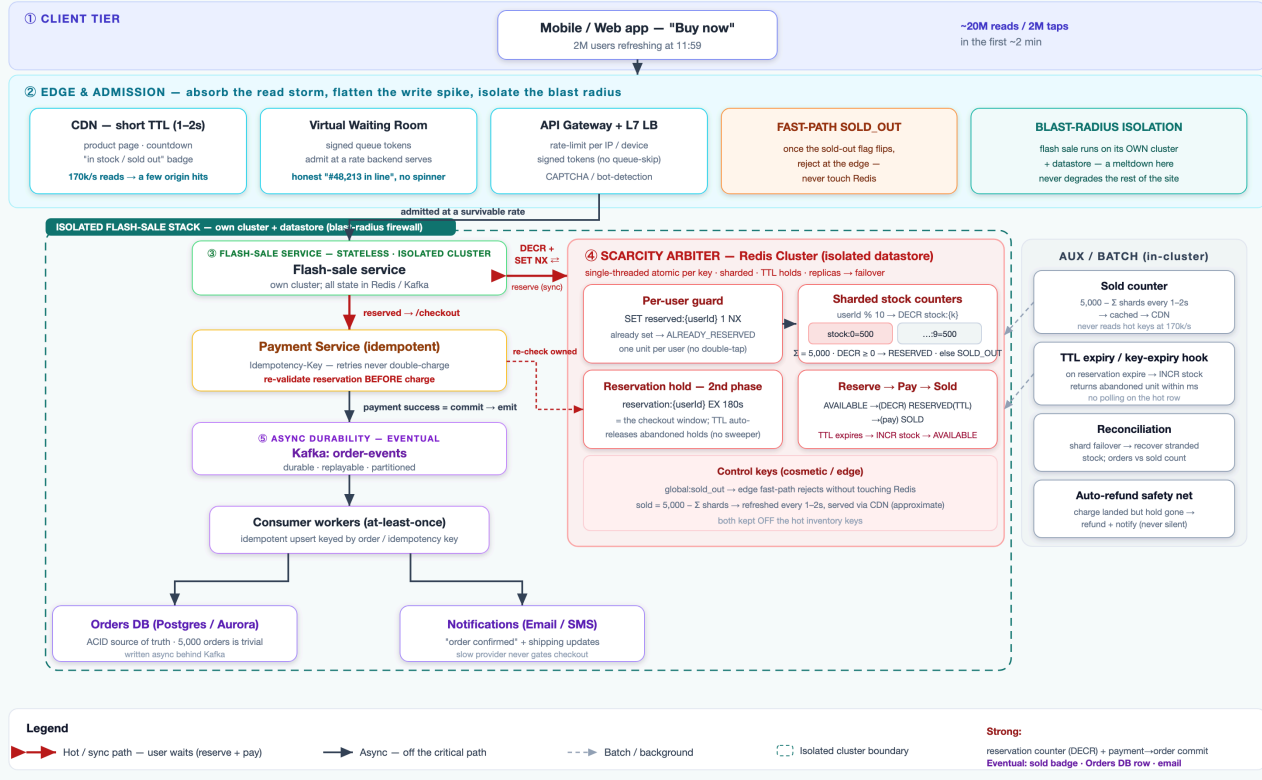
A flash sale is **"sell exactly N scarce units, with a payment step, under a read storm and a write spike, without taking down the platform."** The winning design: **CDN + virtual waiting room** to absorb reads and flatten the write spike; **sharded atomic Redis counters with TTL-based holds** for a never-oversell two-phase reserve→pay; **idempotent payment** as the commit point; **Kafka + relational Orders DB** as the async, ACID source of truth; and **hard isolation** so the drop can melt without hurting the rest of the site. It's the burger giveaway plus a two-phase reservation and real money — same atomic-scarcity backbone, stricter correctness on the pay step.

Flash Sale — Fixed Inventory, Extreme Demand

Sharded atomic Redis counters · two-phase reserve→pay with TTL holds · re-validate-before-charge
· blast-radius isolation · never oversold

Flash Sale — Fixed Inventory, Extreme Demand — Backend Architecture

Sell exactly 5,000 units under a ~170k/s read storm + ~65k/s write spike — never oversell, feel fair, and never take down the rest of the platform.



Flash Sale — Data Model & APIs

Companion to `HLD-Interview-Flash-Sale.pdf`. The transcript reasons through the *design* (CDN + waiting room, sharded atomic counters, two-phase reserve→pay, re-validate-before-charge, blast-radius isolation) but never writes down the **data model** or the **API contract** as artifacts. This fills both; `HLD-Flash-Sale-Architecture.png` / `.svg` carries the full visual architecture.

One-line recap of the design: a CDN plus a virtual waiting room absorb the ~170k/s read storm and flatten the ~65k/s write spike into an admit rate the backend can serve; sharded atomic Redis counters (pre-loaded with the 5,000 units) guarantee we never oversell, with a TTL-based `reservation:{userId}` hold implementing the second phase (reserve → pay → sold); payment is idempotent and re-validates the hold at charge time; a relational Orders DB behind Kafka is the async ACID source of truth; and the whole flash-sale service runs on its **own isolated cluster + datastore** so the drop can melt without touching the rest of the platform.

1. Data model

The system has **three stores**, each authoritative for a different thing — all living inside the **isolated flash-sale cluster**:

Store	Authoritative for	Consistency
Redis cluster (sharded)	live scarcity — remaining stock + the reserve→pay→sold hold	strong / linearizable (per key)
Kafka (order-events)	the durable event log / replay for order fulfilment	durable, ordered per partition
PostgreSQL / Aurora	the permanent, ACID record of every order (money + shipment)	ACID (eventually written)

1.1 Redis keyspace (the hot path — the *only* thing on the critical path)

Key	Type	Init / Value	Purpose	Op on hot path
<code>stock:{k}</code> <code>k = 0..9</code>	Integer (counter)	each set to <code>500</code> ; $\Sigma = 5,000$	sharded inventory — one atomic counter per shard	<code>DECR</code> (reserve if result ≥ 0)
<code>reserved:{userId}</code>	String flag	<code>SET ... NX</code>	per-user dedup guard (one unit per user)	<code>SET NX</code> (proceed only if newly set)
<code>reservation:{userId}</code>	String, EX 180	<code>{shard, ts}</code>	the 2nd-phase hold — the checkout window; self-expiring	<code>SET ... EX</code> ; <code>GET</code> to re-validate; <code>DEL</code> on sold
<code>global:sold_out</code>	String flag	<code>"0"</code> → <code>"1"</code> when all shards empty	edge fast-path: reject at gateway without touching Redis	read at edge; set by aggregator
<code>sold:count</code>	Integer	<code>5000 - Σ stock:{k}</code> , refreshed every 1-2s	cosmetic "3,201 sold" counter (approximate)	read-only (never contends with real inventory)

Key design choices baked into the keyspace - Two keys per reservation, not one. `reserved:{userId}` is the *dedup guard* (idempotent "this user already got one"); `reservation:{userId}` is the *self-expiring hold* whose TTL == the checkout window. Splitting them lets the hold expire and return stock while still remembering the user reserved. - **SET NX before DECR**. Dedup first, decrement second — a retry is free and a double-tap

can't grab two units. - **The TTL is the whole point of the second phase.** Without it, users who reserve but never pay permanently lock units and we "sell out" with unpaid holds — the classic ticketing bug. Redis key expiry returns the unit to the pool within milliseconds, with **no polling sweeper on the hot row** (see the transcript's §10 deep-dive on why a DB decrement can't do this). - **Shard the counter (10 × 500)** to spread the 65k/s write spike across keys, with **sibling-probing** near the end so all 5,000 truly sell. - **On SOLD_OUT we DEL reserved:{userId}** so a user who tapped into an empty shard hasn't burned their one chance.

Atomic sibling-probe (Lua) — check-and-decrement in one shot, no racy flicker

```
-- KEYS = a bounded handful of candidate stock shards to try
-- returns the 1-based index of the shard reserved from, or -1 if all empty
for i = 1, #KEYS do
  local v = tonumber(redis.call('GET', KEYS[i]) or '0')
  if v > 0 then
    redis.call('DECR', KEYS[i])
    return i          -- reserved atomically; no "DECR → see -1 → INCR back" race
  end
end
return -1
```

1.2 Kafka — topic `order-events`

- **Durable, replicated, retained** so it doubles as the replayable order/fulfilment stream. Partitioned (e.g. by `orderId`) for parallel draining.
- **Produced by the flash-sale service, not by Redis.** After Redis reserves and the Payment Service confirms the charge, *the service* — with payment success as the commit point — produces `order-confirmed` (at-least-once) before returning. So an order is never "charged in the gateway but has no record." (Nothing wires Redis directly to Kafka.)

```
// message value (key = orderId)
{
  "orderId":    "ord_01J8ZT...",
  "userId":    "u_88213771",
  "sku":       "iphone-2026",
  "shard":     4,                // which stock shard it came from (reconciliation)
  "amount":    99900,           // cents
  "currency":  "USD",
  "paymentRef": "pi_3Q...gateway",
  "confirmedAt": "2026-07-23T12:00:01.412Z",
  "idempotencyKey": "u_88213771:iphone-2026",
  "eventId":   "01J...ULID"     // idempotency key for the consumer upsert
}
```

1.3 PostgreSQL / Aurora — source of truth (written async by consumers)

```
-- Every confirmed order. 5,000 rows is trivial.
CREATE TABLE orders (
  order_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id       TEXT          NOT NULL,
  sku           TEXT          NOT NULL,
  shard         SMALLINT      NOT NULL,
  amount        INT           NOT NULL,
  currency       TEXT          NOT NULL DEFAULT 'USD',
  status        TEXT          NOT NULL DEFAULT 'CONFIRMED', -- CONFIRMED | SHIPPED | REFUNDED
  payment_ref    TEXT          NOT NULL,                  -- gateway transaction id
  idempotency_key TEXT        NOT NULL,                  -- /checkout Idempotency-Key
  confirmed_at   TIMESTAMPTZ  NOT NULL,
  event_id      TEXT          NOT NULL,                  -- Kafka eventId, idempotent upsert
  CONSTRAINT uq_orders_user_sku UNIQUE (user_id, sku),   -- backstop: one unit per user per dr
  CONSTRAINT uq_orders_idem    UNIQUE (idempotency_key), -- makes /checkout idempotent
  CONSTRAINT uq_orders_event   UNIQUE (event_id)         -- makes the consumer write idempotent
);
CREATE INDEX idx_orders_status ON orders (status, confirmed_at);

-- Idempotency ledger for /checkout (so a retried Pay returns the SAME result).
CREATE TABLE idempotency_keys (
  idempotency_key TEXT PRIMARY KEY,
  user_id         TEXT          NOT NULL,
  result_json     JSONB         NOT NULL,              -- the original response to replay
  created_at      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- Sale configuration / counters snapshot (single row).
CREATE TABLE sale (
  sale_id      TEXT PRIMARY KEY,
  sku          TEXT          NOT NULL,
  total_units  INT           NOT NULL,    -- 5,000
  shard_count  INT           NOT NULL,    -- 10
  opens_at     TIMESTAMPTZ  NOT NULL,
  checkout_ttl_seconds INT NOT NULL DEFAULT 180
);
```

The consumer does an **idempotent upsert** keyed by `event_id` (`INSERT ... ON CONFLICT (event_id) DO NOTHING`), so at-least-once delivery / replay never creates duplicate orders. `UNIQUE(user_id, sku)` and `UNIQUE(idempotency_key)` are *correctness backstops* — the primary scarcity guard is Redis `DECR / SET NX`, which keeps the 65k/s uniqueness check off the database.

Reserve-vs-sold gap: a `RESERVED` unit is neither available nor sold — deliberately treated as "spent" for availability (so we never oversell) but not yet revenue. TTL expiry reconciles the gap by returning abandoned

holds. **Reconciliation invariant:** `COUNT(orders WHERE status != 'REFUNDED') ≤ 5,000`, and `sold = 5,000 - ∑ stock:{k} - (live RESERVED holds)`.

Blast-radius isolation: every store above runs in the **flash-sale cluster's own datastore**, separate from the main commerce stack. If the sale melts under load, the rest of the catalog is untouched — the isolation requirement made concrete.

2. API design

Reads are cacheable and served from the CDN. Write traffic passes the waiting room first and carries an **Idempotency-Key: {userId}:{sku}** so a retried/timed-out tap is free — it resolves to the same outcome and never charges twice or reserves two units.

2.1 `GET /v1/sale/{sku}` — product page / status (cacheable, approximate)

```
// 200 — fully cached at the CDN (1-2s TTL); ~0 origin load under the read storm
{ "sku": "iphone-2026", "status": "LIVE", "opensAt": "2026-07-23T12:00:00Z",
  "soldOut": false, "sold": 3201, "approximate": true }
```

Served entirely from the CDN cache. `sold` reads the pre-computed `sold:count` — **never** sums the live stock shards on the request path.

2.2 `POST /v1/reserve` — reserve one unit (the hot endpoint, phase 1)

Requires a valid **signed waiting-room token**. Idempotent per user.

Request

```
POST /v1/reserve
Idempotency-Key: u_88213771:iphone-2026
Authorization: Bearer <waiting-room queue token>
Content-Type: application/json

{ "userId": "u_88213771", "sku": "iphone-2026" }
```

Server logic (hot path): 1. `SET reserved:{userId} 1 NX` → if it already exists → **ALREADY_RESERVED** (idempotent replay or a second tap). 2. Else `DECR stock:{userId % 10}` → if ≥ 0 → reserved. 3. Else bounded sibling-probe over the remaining shards (Lua) → if one found → reserved. 4. Else `DEL reserved:{userId}` (release the guard) → **SOLD_OUT**. 5. On reserve: write `reservation:{userId} EX 180` (start the checkout window) and return **RESERVED** + a checkout deadline.

Responses (HTTP `200` with a discriminated `result`; status codes in parens for REST purists):

```
// RESERVED (200 / 201)
{ "result": "RESERVED", "reservationId": "rsv_9f2a", "sku": "iphone-2026",
  "expiresAt": "2026-07-23T12:03:00Z", "message": "Reserved – check out within 3 minutes" }

// ALREADY_RESERVED (200 – idempotent replay or a genuine second tap)
{ "result": "ALREADY_RESERVED", "reservationId": "rsv_9f2a", "expiresAt": "..." }

// SOLD_OUT (200, or 410 Gone)
{ "result": "SOLD_OUT", "message": "Sorry, sold out" }
```

Failure	Status	Meaning
No / invalid waiting-room token	401	must pass through the waiting room first
Rate limit (per IP / device)	429	bot / hammering; <code>Retry-After</code> header
Backend saturated (admission control)	503	{ "result": "BUSY" } – graceful valve
Sale not open / closed	409	outside [opens_at, ...]

Fast-path SOLD_OUT: once `global:sold_out = 1`, the gateway/CDN short-circuits and returns `SOLD_OUT` at the edge **without touching Redis** — the graceful-degradation valve for the 1.995M who lose.

2.3 `POST /v1/checkout` — pay & confirm (phase 2, re-validate before charge)

The one place a TTL race could otherwise oversell or charge-without-item. **Order of operations gates the charge on the hold.**

Request

```
POST /v1/checkout
Idempotency-Key: u_88213771:iphone-2026
Authorization: Bearer <session>

{ "userId": "u_88213771", "reservationId": "rsv_9f2a", "payment": { ... } }
```

Server logic: 1. If this `Idempotency-Key` is already in the ledger → **replay the stored result** (no re-charge). 2. `GET reservation:{userId}` — if missing / not owned by this user → **RESERVATION_EXPIRED** (no money moves). (Optionally promote to a `PAYMENT_IN_PROGRESS` state with a fresh TTL so an actively-paying user's hold can't expire under them.) 3. Only if the hold is still ours: `paymentSvc.capture(idempotencyKey)`. 4. On success: emit `order-confirmed` to Kafka (payment success = the commit point), store the idempotency result, `DEL reservation:{userId}` + mark the shard's unit `SOLD`; return confirmation. On failure: `INCR stock:{shard}` (release), tell the user. 5. **Safety net:** if the charge landed but the hold had flipped in the residual window → **auto-refund** + notify (never silently keep the money).

Responses:

```
// CONFIRMED (200)
{ "result": "CONFIRMED", "orderId": "ord_01J8ZT...", "message": "Order confirmed! 🎉" }

// RESERVATION_EXPIRED (200, or 409) – hold lapsed; NEVER charged
{ "result": "RESERVATION_EXPIRED", "message": "Your hold expired – the unit was released" }

// PAYMENT_FAILED (402) – hold released, unit returned to pool
{ "result": "PAYMENT_FAILED", "message": "Payment failed – please try again" }
```

Why re-check right before capture: the reservation TTL, the user's typing speed, and the gateway's response time are three independent clocks. The invariant is **never take money without a freshly validated hold** — re-checking `reservation:{userId} == userId` as the first step of `/checkout` is what actually prevents oversell and charge-without-item; a longer TTL and a payment-start renewal only make the race rarer. The unique constraint is the last-line DB backstop, but it should never be the thing that catches a bad charge.

2.4 GET /v1/reservation/status?userId= — check my reservation (idempotent read)

```
// 200
{ "userId": "u_88213771", "status": "RESERVED", "expiresAt": "2026-07-23T12:03:00Z" }
// or { "status": "SOLD" } / { "status": "NONE" } / { "status": "EXPIRED" }
```

Reads Redis (`reservation:{userId}`), falls back to Postgres (`orders`) for post-sale lookups. Kept off the inventory keys.

2.5 Admin / ops (internal, authenticated)

Endpoint	Purpose
GET /admin/reconcile	run/report the <code>orders</code> vs Redis stock reconciliation (<code>orders ≤ 5,000</code>)
GET /admin/inventory	per-shard remaining + <code>sold_out</code> flag for the on-call dashboard
POST /admin/sale/open /close	flip the sale window manually

3. Request lifecycle (happy path)

The **flash-sale service is the orchestrator**: it makes synchronous request/response calls to Redis (reserve) and the Payment Service (capture), and it is the Kafka **producer**. Redis never talks to Kafka. Everything below runs inside the **isolated flash-sale cluster**.


```

Client → CDN (read storm absorbed) → Waiting room (signed token) → Gateway (rate-limit, admit)
→ Flash-sale service
  ├──(1) ⇌ Redis: SET NX → DECR stock → SET reservation EX 180 ..... sync → RESERVED
  |   "reserved, pay within 3 min" ..... user enters card
  ├──(2) ⇌ Redis: GET reservation == userId (re-validate) ..... BEFORE charge
  ├──(3) → Payment Service: capture(idempotencyKey) ..... only if hold still
  ├──(4) → returns "order confirmed" to the user ..... user is done here
  └──(5) → Kafka (order-events) ..... async, produced later
        → Consumer workers
          ├── Orders DB (ACID source of truth, idempotent upsert)
          └── Email/SMS "order confirmed" + fulfilment

```

The user gets "**order confirmed**" the instant payment + reservation validate (step 3→4). The durable order write and notification (step 5) are produced to Kafka and catch up behind it — they never touch the user's latency, and a slow warehouse/SMS provider never gates checkout. The whole reserve path is memory-speed: Redis atomic ops, no synchronous DB write until the rarer, latency-tolerant order persist.